

```
#prn:23070521126 SANIYA GOUTAM
import numpy as np
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding
from tensorflow.keras.layers import LSTM
from tensorflow.keras.layers import Dense
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from sklearn.model_selection import train_test_split
```

```
sentences = [
    "book a flight from mumbai to delhi",
    "book a ticket from nagpur to pune",
    "find a flight from delhi to bangalore",
    "travel from mumbai to chennai",
    "flight from pune to hyderabad",
    "book flight from chennai to mumbai",
    "i want to fly from nagpur to delhi",
    "find ticket from bangalore to mumbai",
    "book a flight from hyderabad to pune",
    "travel from delhi to chennai",
    "i need a flight from pune to delhi",
    "book ticket from mumbai to nagpur",
    "find flight from chennai to bangalore",
    "travel from hyderabad to mumbai",
    "book a flight from delhi to pune",
    "i want to travel from nagpur to mumbai",
    "find a ticket from pune to chennai",
    "book flight from bangalore to delhi",
    "travel from mumbai to hyderabad",
    "find flight from chennai to pune"
]
```

```
labels = [
    "0 0 0 0 B-from_city 0 B-to_city",
    "0 0 0 0 B-from_city 0 B-to_city",
    "0 0 0 0 B-from_city 0 B-to_city",
    "0 0 0 B-from_city 0 B-to_city",
    "0 0 B-from_city 0 B-to_city",
    "0 0 0 B-from_city 0 B-to_city",
    "0 0 0 0 B-from_city 0 B-to_city",
    "0 0 0 B-from_city 0 B-to_city",
    "0 0 0 0 B-from_city 0 B-to_city",
    "0 0 B-from_city 0 B-to_city",
    "0 0 0 0 B-from_city 0 B-to_city",
    "0 0 0 B-from_city 0 B-to_city",
    "0 0 B-from_city 0 B-to_city",
    "0 0 0 0 B-from_city 0 B-to_city",
    "0 0 0 0 B-from_city 0 B-to_city",
    "0 0 0 0 B-from_city 0 B-to_city",
    "0 0 0 B-from_city 0 B-to_city",
    "0 0 B-from_city 0 B-to_city",
    "0 0 0 B-from_city 0 B-to_city"
]
```

```
# convert words into numbers.

tokenizer = Tokenizer()

tokenizer.fit_on_texts(sentences)

X = tokenizer.texts_to_sequences(sentences)
```

```
label_to_id = {
    "0": 0,
    "B-from_city": 1,
    "B-to_city": 2
}
```

```

y = []

for label_sentence in labels:

    label_ids = [
        label_to_id[label]
        for label in label_sentence.split()
    ]

    y.append(label_ids)

```

```

max_len = max(len(seq) for seq in X)

```

```

X = pad_sequences(
    X,
    maxlen=max_len,
    padding='post'
)

```

```

y = pad_sequences(
    y,
    maxlen=max_len,
    padding='post',
    value=0
)

```

```

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

```

```

# LSTM model

vocab_size = len(tokenizer.word_index) + 1

num_labels = len(label_to_id)

model = Sequential()

model.add(
    Embedding(
        input_dim=vocab_size,
        output_dim=32,
        input_length=max_len
    )
)

model.add(
    LSTM(
        64,
        return_sequences=True
    )
)

model.add(
    Dense(
        num_labels,
        activation='softmax'
    )
)

```

```

/usr/local/lib/python3.13/dist-packages/keras/src/layers/core/embedding.py:100: UserWarning: Argument `input_length` is deprecated
warnings.warn(

```

```

model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

```

```

history = model.fit(
    X_train,
    y_train,
    epochs=100,
    batch_size=4,
    validation_split=0.2,
    verbose=1
)

```

```

Epoch 72/100
3/3 ————— 0s 28ms/step - accuracy: 0.9583 - loss: 0.0986 - val_accuracy: 0.9688 - val_loss: 0.2397
Epoch 73/100
3/3 ————— 0s 28ms/step - accuracy: 0.9583 - loss: 0.1043 - val_accuracy: 0.9688 - val_loss: 0.2462
Epoch 74/100
3/3 ————— 0s 29ms/step - accuracy: 0.9583 - loss: 0.0988 - val_accuracy: 0.9688 - val_loss: 0.2376
Epoch 75/100
3/3 ————— 0s 27ms/step - accuracy: 0.9583 - loss: 0.0942 - val_accuracy: 0.9688 - val_loss: 0.2325
Epoch 76/100
3/3 ————— 0s 29ms/step - accuracy: 0.9583 - loss: 0.0912 - val_accuracy: 0.9688 - val_loss: 0.2360
Epoch 77/100
3/3 ————— 0s 27ms/step - accuracy: 0.9583 - loss: 0.0877 - val_accuracy: 0.9688 - val_loss: 0.2423
Epoch 78/100
3/3 ————— 0s 32ms/step - accuracy: 0.9583 - loss: 0.0885 - val_accuracy: 0.9688 - val_loss: 0.2411
Epoch 79/100
3/3 ————— 0s 35ms/step - accuracy: 0.9583 - loss: 0.0849 - val_accuracy: 0.9688 - val_loss: 0.2394
Epoch 80/100
3/3 ————— 0s 28ms/step - accuracy: 0.9688 - loss: 0.0834 - val_accuracy: 0.9688 - val_loss: 0.2384
Epoch 81/100
3/3 ————— 0s 29ms/step - accuracy: 0.9583 - loss: 0.0814 - val_accuracy: 0.9688 - val_loss: 0.2405
Epoch 82/100
3/3 ————— 0s 29ms/step - accuracy: 0.9583 - loss: 0.0791 - val_accuracy: 0.9688 - val_loss: 0.2377
Epoch 83/100
3/3 ————— 0s 28ms/step - accuracy: 0.9583 - loss: 0.0779 - val_accuracy: 0.9688 - val_loss: 0.2352
Epoch 84/100
3/3 ————— 0s 28ms/step - accuracy: 0.9688 - loss: 0.0768 - val_accuracy: 0.9688 - val_loss: 0.2388
Epoch 85/100
3/3 ————— 0s 28ms/step - accuracy: 0.9583 - loss: 0.0737 - val_accuracy: 0.9688 - val_loss: 0.2419
Epoch 86/100
3/3 ————— 0s 27ms/step - accuracy: 0.9583 - loss: 0.0744 - val_accuracy: 0.9688 - val_loss: 0.2435
Epoch 87/100
3/3 ————— 0s 29ms/step - accuracy: 0.9583 - loss: 0.0748 - val_accuracy: 0.9688 - val_loss: 0.2393
Epoch 88/100
3/3 ————— 0s 29ms/step - accuracy: 0.9688 - loss: 0.0730 - val_accuracy: 0.9688 - val_loss: 0.2400
Epoch 89/100
3/3 ————— 0s 28ms/step - accuracy: 0.9583 - loss: 0.0688 - val_accuracy: 0.9688 - val_loss: 0.2462
Epoch 90/100
3/3 ————— 0s 27ms/step - accuracy: 0.9583 - loss: 0.0680 - val_accuracy: 0.9688 - val_loss: 0.2520
Epoch 91/100
3/3 ————— 0s 35ms/step - accuracy: 0.9583 - loss: 0.0729 - val_accuracy: 0.9688 - val_loss: 0.2456
Epoch 92/100
3/3 ————— 0s 29ms/step - accuracy: 0.9583 - loss: 0.0694 - val_accuracy: 0.9688 - val_loss: 0.2429
Epoch 93/100
3/3 ————— 0s 30ms/step - accuracy: 0.9688 - loss: 0.0634 - val_accuracy: 0.9688 - val_loss: 0.2443
Epoch 94/100
3/3 ————— 0s 27ms/step - accuracy: 0.9688 - loss: 0.0622 - val_accuracy: 0.9688 - val_loss: 0.2459
Epoch 95/100
3/3 ————— 0s 27ms/step - accuracy: 0.9583 - loss: 0.0654 - val_accuracy: 0.9688 - val_loss: 0.2438
Epoch 96/100
3/3 ————— 0s 33ms/step - accuracy: 0.9688 - loss: 0.0601 - val_accuracy: 0.9688 - val_loss: 0.2402
Epoch 97/100
3/3 ————— 0s 32ms/step - accuracy: 0.9688 - loss: 0.0596 - val_accuracy: 0.9688 - val_loss: 0.2403
Epoch 98/100
3/3 ————— 0s 30ms/step - accuracy: 0.9583 - loss: 0.0650 - val_accuracy: 0.9688 - val_loss: 0.2417
Epoch 99/100
3/3 ————— 0s 28ms/step - accuracy: 0.9583 - loss: 0.0593 - val_accuracy: 0.9688 - val_loss: 0.2483
Epoch 100/100
3/3 ————— 0s 31ms/step - accuracy: 0.9583 - loss: 0.0732 - val_accuracy: 0.9688 - val_loss: 0.2528

```

```

loss, accuracy = model.evaluate(
    X_test,
    y_test,
    verbose=0
)

```

```
print("Test Accuracy:", accuracy)
```

```
Test Accuracy: 1.0
```

```

# Test a New Sentence
test_sentence = "book a flight from mumbai to pune"

```

```
test_sequence = tokenizer.texts_to_sequences(
    [test_sentence]
)
```

```
test_sequence = pad_sequences(
    test_sequence,
    maxlen=max_len,
    padding='post'
)
```

```
prediction = model.predict(
    test_sequence
)
```

1/1  0s 139ms/step

```
predicted_ids = np.argmax(
    prediction,
    axis=-1
)[0]
```

```
id_to_label = {
    0: "0",
    1: "B-from_city",
    2: "B-to_city"
}
```

```
words = test_sentence.split()
```

```
for word, label_id in zip(
    words,
    predicted_ids
):
    print(
        word,
        "->",
        id_to_label[label_id]
    )
```

```
book -> 0
a -> 0
flight -> 0
from -> 0
mumbai -> B-from_city
to -> 0
pune -> B-to_city
```

```
import numpy as np

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding
from tensorflow.keras.layers import LSTM
from tensorflow.keras.layers import Dense

from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences

from sklearn.model_selection import train_test_split

# -----
# 1. Create Dataset
# -----

sentences = [
    "book a flight from mumbai to delhi",
    "book a ticket from nagpur to pune",
    "find a flight from delhi to bangalore",
    "travel from mumbai to chennai",
    "flight from pune to hyderabad",
    "book flight from chennai to mumbai",
    "i want to fly from nagpur to delhi",
    "find ticket from bangalore to mumbai"
```

```

"book a flight from hyderabad to pune",
"travel from delhi to chennai",
"i need a flight from pune to delhi",
"book ticket from mumbai to nagpur",
"find flight from chennai to bangalore",
"travel from hyderabad to mumbai",
"book a flight from delhi to pune",
"i want to travel from nagpur to mumbai",
"find a ticket from pune to chennai",
"book flight from bangalore to delhi",
"travel from mumbai to hyderabad",
"find flight from chennai to pune"
]

# -----
# 2. Slot Labels
# -----

labels = [
    "0 0 0 0 B-from_city 0 B-to_city",
    "0 0 0 0 B-from_city 0 B-to_city",
    "0 0 0 0 B-from_city 0 B-to_city",
    "0 0 0 B-from_city 0 B-to_city",
    "0 0 B-from_city 0 B-to_city",
    "0 0 0 B-from_city 0 B-to_city",
    "0 0 0 0 B-from_city 0 B-to_city",
    "0 0 0 B-from_city 0 B-to_city",
    "0 0 0 0 B-from_city 0 B-to_city",
    "0 0 B-from_city 0 B-to_city",
    "0 0 0 0 B-from_city 0 B-to_city",
    "0 0 0 B-from_city 0 B-to_city",
    "0 0 0 B-from_city 0 B-to_city",
    "0 0 B-from_city 0 B-to_city",
    "0 0 0 0 B-from_city 0 B-to_city",
    "0 0 0 0 B-from_city 0 B-to_city",
    "0 0 0 B-from_city 0 B-to_city",
    "0 0 B-from_city 0 B-to_city",
    "0 0 0 B-from_city 0 B-to_city"
]

# -----
# 3. Tokenization
# -----

tokenizer = Tokenizer()

tokenizer.fit_on_texts(sentences)

X = tokenizer.texts_to_sequences(sentences)

# -----
# 4. Encode Labels
# -----

label_to_id = {
    "0": 0,
    "B-from_city": 1,
    "B-to_city": 2
}

y = []

for label_sentence in labels:
    label_ids = [
        label_to_id[label]
        for label in label_sentence.split()
    ]

    y.append(label_ids)

# -----

```

```

# 5. Padding
# -----

max_len = max(
    len(seq)
    for seq in X
)

X = pad_sequences(
    X,
    maxlen=max_len,
    padding='post'
)

y = pad_sequences(
    y,
    maxlen=max_len,
    padding='post',
    value=0
)

# -----
# 6. Train-Test Split
# -----

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

# -----
# 7. Build LSTM Model
# -----

vocab_size = len(
    tokenizer.word_index
) + 1

num_labels = len(
    label_to_id
)

model = Sequential()

model.add(
    Embedding(
        input_dim=vocab_size,
        output_dim=32,
        input_length=max_len
    )
)

model.add(
    LSTM(
        64,
        return_sequences=True
    )
)

model.add(
    Dense(
        num_labels,
        activation='softmax'
    )
)

# -----
# 8. Compile Model
# -----

model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

```

```

loss=sparse_categorical_crossentropy,
metrics=['accuracy']
)

# -----
# 9. Display Model
# -----

model.summary()

# -----
# 10. Train Model
# -----

history = model.fit(
    X_train,
    y_train,
    epochs=100,
    batch_size=4,
    validation_split=0.2,
    verbose=1
)

# -----
# 11. Evaluate Model
# -----

loss, accuracy = model.evaluate(
    X_test,
    y_test,
    verbose=0
)

print(
    "\nTest Accuracy:",
    accuracy
)

# -----
# 12. Test New Sentence
# -----

test_sentence = (
    "book a flight from mumbai to pune"
)

test_sequence = tokenizer.texts_to_sequences(
    [test_sentence]
)

test_sequence = pad_sequences(
    test_sequence,
    maxlen=max_len,
    padding='post'
)

# -----
# 13. Prediction
# -----

prediction = model.predict(
    test_sequence,
    verbose=0
)

predicted_ids = np.argmax(
    prediction,
    axis=-1
)[0]

# -----
# 14. Convert IDs to Labels

```

```
# -----  
  
id_to_label = {  
    0: "0",  
    1: "B-from_city",  
    2: "B-to_city"  
}  
  
words = test_sentence.split()  
  
print("\nSlot Filling Result:")  
  
for word, label_id in zip(  
    words,  
    predicted_ids  
):  
  
    print(  
        word,  
        "->",  
        id_to_label[label_id]  
    )
```


Model: "sequential_1"

Layer (type)	Output Shape	Param #
embedding_1 (Embedding)	?	0 (unbuilt)
lstm_1 (LSTM)	?	0 (unbuilt)
dense_1 (Dense)	?	0 (unbuilt)

Total params: 0 (0.00 B)

Trainable params: 0 (0.00 B)

Non-trainable params: 0 (0.00 B)

Epoch 1/100

3/3 ————— 2s 181ms/step - accuracy: 0.5208 - loss: 1.0947 - val_accuracy: 0.7188 - val_loss: 1.0881

Epoch 2/100

3/3 ————— 0s 36ms/step - accuracy: 0.7396 - loss: 1.0761 - val_accuracy: 0.7500 - val_loss: 1.0730

Epoch 3/100

3/3 ————— 0s 39ms/step - accuracy: 0.7500 - loss: 1.0545 - val_accuracy: 0.7500 - val_loss: 1.0555

Epoch 4/100

3/3 ————— 0s 38ms/step - accuracy: 0.7500 - loss: 1.0290 - val_accuracy: 0.7500 - val_loss: 1.0348

Epoch 5/100

3/3 ————— 0s 42ms/step - accuracy: 0.7500 - loss: 0.9962 - val_accuracy: 0.7500 - val_loss: 1.0103

Epoch 6/100

3/3 ————— 0s 34ms/step - accuracy: 0.7500 - loss: 0.9555 - val_accuracy: 0.7500 - val_loss: 0.9827

Epoch 7/100

3/3 ————— 0s 57ms/step - accuracy: 0.7500 - loss: 0.9050 - val_accuracy: 0.7500 - val_loss: 0.9564

Epoch 8/100

3/3 ————— 0s 45ms/step - accuracy: 0.7500 - loss: 0.8526 - val_accuracy: 0.7500 - val_loss: 0.9452

Epoch 9/100

3/3 ————— 0s 41ms/step - accuracy: 0.7500 - loss: 0.8240 - val_accuracy: 0.7500 - val_loss: 0.9661

Epoch 10/100

3/3 ————— 0s 58ms/step - accuracy: 0.7500 - loss: 0.8260 - val_accuracy: 0.7500 - val_loss: 0.9821

Epoch 11/100

3/3 ————— 0s 43ms/step - accuracy: 0.7500 - loss: 0.8243 - val_accuracy: 0.7500 - val_loss: 0.9591

Epoch 12/100

3/3 ————— 0s 41ms/step - accuracy: 0.7500 - loss: 0.8062 - val_accuracy: 0.7500 - val_loss: 0.9296

Epoch 13/100

3/3 ————— 0s 29ms/step - accuracy: 0.7500 - loss: 0.7898 - val_accuracy: 0.7500 - val_loss: 0.9086

Epoch 14/100

3/3 ————— 0s 29ms/step - accuracy: 0.7500 - loss: 0.7793 - val_accuracy: 0.7500 - val_loss: 0.8957

Epoch 15/100

3/3 ————— 0s 28ms/step - accuracy: 0.7500 - loss: 0.7715 - val_accuracy: 0.7500 - val_loss: 0.8855

Epoch 16/100

3/3 ————— 0s 28ms/step - accuracy: 0.7500 - loss: 0.7620 - val_accuracy: 0.7500 - val_loss: 0.8750

Epoch 17/100

3/3 ————— 0s 28ms/step - accuracy: 0.7500 - loss: 0.7498 - val_accuracy: 0.7500 - val_loss: 0.8644

Epoch 18/100

3/3 ————— 0s 29ms/step - accuracy: 0.7500 - loss: 0.7354 - val_accuracy: 0.7500 - val_loss: 0.8526

Epoch 19/100

3/3 ————— 0s 52ms/step - accuracy: 0.7500 - loss: 0.7207 - val_accuracy: 0.7500 - val_loss: 0.8403

Epoch 20/100

3/3 ————— 0s 29ms/step - accuracy: 0.7500 - loss: 0.7065 - val_accuracy: 0.7500 - val_loss: 0.8254

Epoch 21/100

3/3 ————— 0s 29ms/step - accuracy: 0.7500 - loss: 0.6887 - val_accuracy: 0.7500 - val_loss: 0.8077

Epoch 22/100

3/3 ————— 0s 27ms/step - accuracy: 0.7500 - loss: 0.6710 - val_accuracy: 0.7500 - val_loss: 0.7882

Epoch 23/100

3/3 ————— 0s 27ms/step - accuracy: 0.7500 - loss: 0.6497 - val_accuracy: 0.7500 - val_loss: 0.7678

Epoch 24/100

3/3 ————— 0s 28ms/step - accuracy: 0.7500 - loss: 0.6293 - val_accuracy: 0.7500 - val_loss: 0.7458

Epoch 25/100

3/3 ————— 0s 27ms/step - accuracy: 0.7500 - loss: 0.6068 - val_accuracy: 0.7500 - val_loss: 0.7222

Epoch 26/100

3/3 ————— 0s 28ms/step - accuracy: 0.7500 - loss: 0.5829 - val_accuracy: 0.7500 - val_loss: 0.6971

Epoch 27/100

3/3 ————— 0s 28ms/step - accuracy: 0.7604 - loss: 0.5573 - val_accuracy: 0.7812 - val_loss: 0.6699

Epoch 28/100

3/3 ————— 0s 28ms/step - accuracy: 0.8021 - loss: 0.5299 - val_accuracy: 0.7812 - val_loss: 0.6411

Epoch 29/100

3/3 ————— 0s 27ms/step - accuracy: 0.8021 - loss: 0.5025 - val_accuracy: 0.8438 - val_loss: 0.6100

Epoch 30/100

3/3 ————— 0s 28ms/step - accuracy: 0.8125 - loss: 0.4739 - val_accuracy: 0.8750 - val_loss: 0.5764

Epoch 31/100

3/3 ————— 0s 27ms/step - accuracy: 0.8542 - loss: 0.4433 - val_accuracy: 0.8750 - val_loss: 0.5417

Epoch 32/100

3/3 ————— 0s 26ms/step - accuracy: 0.8542 - loss: 0.4133 - val_accuracy: 0.8750 - val_loss: 0.5036

Epoch 33/100

3/3 ————— 0s 28ms/step - accuracy: 0.8542 - loss: 0.3834 - val_accuracy: 0.8750 - val_loss: 0.4632

Epoch 34/100

3/3 ————— 0s 28ms/step - accuracy: 0.8646 - loss: 0.3472 - val_accuracy: 0.8750 - val_loss: 0.4234

Epoch 35/100

3/3 ————— 0s 28ms/step - accuracy: 0.9062 - loss: 0.3138 - val_accuracy: 0.9375 - val_loss: 0.3832

Epoch 36/100

3/3 ————— 0s 27ms/step - accuracy: 0.9479 - loss: 0.2793 - val_accuracy: 0.9688 - val_loss: 0.3452

```
Epoch 37/100
3/3 ————— 0s 27ms/step - accuracy: 0.9583 - loss: 0.2493 - val_accuracy: 0.9688 - val_loss: 0.3098
Epoch 38/100
3/3 ————— 0s 30ms/step - accuracy: 0.9583 - loss: 0.2222 - val_accuracy: 0.9688 - val_loss: 0.2805
Epoch 39/100
3/3 ————— 0s 28ms/step - accuracy: 0.9583 - loss: 0.2028 - val_accuracy: 0.9688 - val_loss: 0.2594
Epoch 40/100
3/3 ————— 0s 28ms/step - accuracy: 0.9583 - loss: 0.1832 - val_accuracy: 0.9688 - val_loss: 0.2365
Epoch 41/100
3/3 ————— 0s 35ms/step - accuracy: 0.9583 - loss: 0.1781 - val_accuracy: 0.9688 - val_loss: 0.2214
Epoch 42/100
3/3 ————— 0s 28ms/step - accuracy: 0.9583 - loss: 0.1682 - val_accuracy: 0.9688 - val_loss: 0.2150
Epoch 43/100
3/3 ————— 0s 28ms/step - accuracy: 0.9583 - loss: 0.1666 - val_accuracy: 0.9688 - val_loss: 0.2066
Epoch 44/100
3/3 ————— 0s 28ms/step - accuracy: 0.9583 - loss: 0.1619 - val_accuracy: 0.9688 - val_loss: 0.2021
```